

Final Report Draft

SQL Injection-Based Authentication Bypass in a Self-Built Login Module with Parameterized Query Mitigation

1. Abstract

This project demonstrates how unsafe SQL query construction in a login module can lead to authentication bypass, and how parameterized queries prevent the same behavior. Two separate authentication flows were implemented in a self-built Python Flask application: a vulnerable flow using string-concatenated SQL, and a secure flow using parameterized binding. Additional hardening controls were included in the secure flow: bcrypt password hashing, account lockout after repeated failures, and audit logging. A 17-row test matrix was executed to compare vulnerable and secure behavior under identical inputs, including normal login cases, crafted SQLi-style payload cases, and edge cases. Results show bypass behavior in the vulnerable flow and consistent blocking in the secure flow, while preserving valid login behavior.

2. Introduction

SQL injection is a vulnerability class where user input is interpreted as SQL syntax instead of plain data. In authentication modules, this can permit unauthorized access if input is concatenated directly into query strings. Because authentication is the entry boundary for protected resources, bypass vulnerabilities are high impact.

This assignment focuses on a controlled academic demonstration in local environment only. The goal is to compare insecure and secure implementation styles directly and explain why parameterized query execution is the correct mitigation.

3. Objectives

1. Implement a vulnerable login flow that uses direct SQL string concatenation.
2. Implement a secure login flow that uses parameterized query binding.
3. Add practical hardening controls: bcrypt hashing, lockout, and logging.
4. Execute and document comparative tests with reproducible evidence.

4. Ethical Scope Statement

- All testing was performed on local machine only.
- Only test users and test database were used.
- No third-party/public/live systems were targeted.
- The vulnerable module exists only for controlled academic demonstration.

5. Literature Survey

SQL injection has remained a prominent web security issue for decades and is included under OWASP A03:2021 Injection. Common forms include in-band/classic SQLi, union-based SQLi, error-based SQLi, and blind/time-based SQLi.

Authentication bypass is especially critical because it may grant access without valid credentials. Industry guidance consistently recommends parameterized queries as the primary defense. Input validation, error handling, least-privilege database configuration, and monitoring are additional controls.

6. System Design

6.1 Architecture

- Frontend pages: home, vulnerable login, secure login, success/fail views.
- Backend routes: `/`, `/login\_vuln`, `/login\_safe`.

- Data layer: SQLite database with `users` and `audit\_log` tables.
- Security utility: logger writes to both `auth.log` and `audit\_log`.

6.2 Data Flow Summary

1. User submits credentials to chosen route.
2. Vulnerable route builds SQL directly from input.
3. Secure route uses placeholder binding and bcrypt verification.
4. Secure route updates lockout counters and emits log events.
5. UI displays result and mode-specific explanation.

7. Implementation - Module A (Vulnerable Flow)

File: `auth\_vulnerable.py`

The vulnerable module constructs SQL using direct string concatenation:

- Username and password are inserted into SQL text.
- Query executes as built string.
- If returned row exists, login is accepted.

This behavior is intentionally insecure and present only for demonstration. The design shows how mixing user input with SQL code can alter authentication logic.

8. Implementation - Module B (Secure Flow)

File: `auth\_secure.py`

The secure module performs authentication in safe stages:

1. Parameterized lookup: `SELECT \* FROM users WHERE username = ?`
2. Generic failure for unknown users.
3. Lockout check using `lock\_until`.
4. Password verification via `bcrypt.checkpw` against stored hash.
5. Reset counters on success.

Key mitigation: SQL structure is parsed/compiled first, and user input is bound later as data. Therefore input cannot modify SQL grammar.

9. Implementation - Module C (Hardening)

9.1 bcrypt Password Hashing

- Password hashes are stored in `password\_hash`.
- Secure flow never compares plaintext passwords.
- Hash records show bcrypt prefix (`\$2b\$`).

9.2 Lockout Logic

- `failed\_attempts` increments on wrong password.
- After threshold (`MAX\_ATTEMPTS = 5`), account is locked for 300 seconds.
- During lock, authentication is denied and lock message is shown.

9.3 Logging

File: `logger.py`

- Events written to `auth.log` and `audit\_log` table.
- Logged fields: timestamp, username, event, source.
- Plaintext password values are never logged.

10. Testing and Results

10.1 Test Matrix Execution

A complete 17-row matrix was executed and documented.

Result file:

- `test\_matrix\_results\_step11\_17rows.md`

Summary:

- Total cases: 17
- Passed: 17
- Failed: 0

10.2 Comparative Outcome

- Vulnerable flow: bypass behavior observed for crafted SQLi-style inputs.
- Secure flow: same inputs were blocked.
- Valid credentials: continued to work in secure flow.
- Lockout and logging: functioned as expected.

10.3 Evidence Files

Evidence screenshots were captured in:

- `evidence/ev01\_normal\_vuln.png`
- `evidence/ev02\_normal\_secure.png`
- `evidence/ev03\_attack\_bypass.png`
- `evidence/ev04\_attack\_blocked.png`
- `evidence/ev05\_comment\_attack.png`
- `evidence/ev06\_comment\_blocked.png`
- `evidence/ev07\_lockout.png`
- `evidence/ev08\_audit\_log.png`
- `evidence/ev09\_code\_vuln.png`
- `evidence/ev10\_code\_secure.png`
- `evidence/ev11\_bcrypt\_hash.png`

11. Discussion

This project reinforces that parameterization is the primary SQL injection defense because it enforces code/data separation at query execution level. Input validation can reduce noise but cannot replace parameterized execution.

The vulnerable/secure side-by-side design helped isolate and demonstrate the effect clearly. Hardening controls addressed related risks: credential storage compromise (hashing), brute-force attempts (lockout), and observability (logging).

Limitations:

- The app is an academic demo using SQLite and development server.
- Production deployment would require hardened infrastructure and secure secret management.

12. Best Practices and Future Work

12.1 Best Practices

- Use parameterized queries everywhere.
- Apply least-privilege DB permissions.
- Use generic error messages.
- Protect credentials with strong hashing.
- Add centralized logging/monitoring.

12.2 Future Work

- Add MFA and CAPTCHA after repeated failures.
- Add role-based authorization checks.
- Add SIEM integration for anomaly detection.
- Add HTTPS deployment and secrets management for production-like setup.

13. Conclusion and References

The assignment objective was achieved: an insecure login flow demonstrated authentication

bypass behavior, and a secure parameterized flow blocked the same behavior while preserving valid authentication. Additional hardening controls improved overall security posture and demonstrability.

References

1. OWASP Top 10 2021 - A03 Injection
2. OWASP SQL Injection Prevention Cheat Sheet
3. Python `sqlite3` documentation
4. `bcrypt` documentation
5. Course material and textbook sections on software security and injection vulnerabilities